



ASSIGNMENT

TECHNOLOGY PARK MALAYSIA

CT073-3-2

Computer Systems Low Level Techniques

APD2F2506CS(AI)

HAND OUT DATE: 19 DECEMBER 2025

HAND IN DATE: 16 MARCH 2026

WEIGHTAGE: 60%

STUDENT NAME: LOO WEN BIN

TP NUMBER: TP081133

PROJECT TITLE:

SIMPLE INVOICE AND RECEIPT MANAGEMENT SYSTEM

Table of Contents

1.0	Introduction on Assembly Language.....	3
2.0	Research and Analysis.....	4
2.1	Malware Analysis.....	4
2.2	Reverse Engineering.....	4
3.0	System Design using flowchart	6
4.0	System Screenshot.....	7
5.0	Source Code with explanation	11
6.0	Conclusion.....	15
	References	15

1.0 Introduction on Assembly Language

Assembly language is a type of low-level language which is more similar to machine language, not like other high-level language like Python closer to human language. It means that the languages are designed to communicate with the computer CPU using instructions. It uses register as the storage unit for instruction and system call service using ASCII table. For this assignment, NASM with 64 bit is chosen to design the system which is simple invoice and receipt management system. Due to limitation of NASM, the system will only be restricted to simple functionality like generating invoices and receipts.

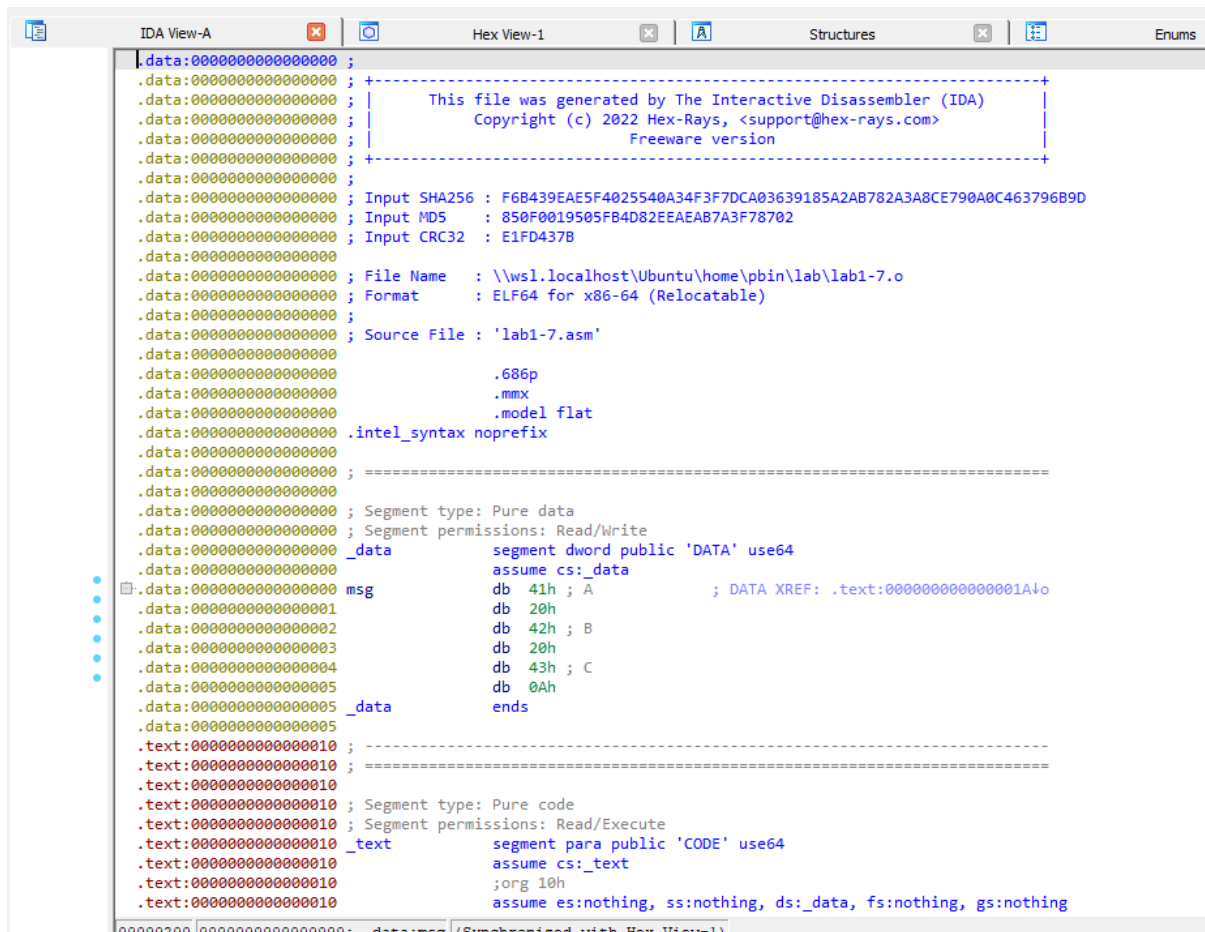
2.0 Research and Analysis

2.1 Malware Analysis

This paper discuss why and how low level language still active in modern cybersecurity environment despite the advancement and common of high level language (Attari & Nawaz, 2024). Assembly language as a level language are needed during process of malware analyst using reverse engineering. It also is the bridge from user to hardware and CPU and was needed while diving deep into deeper layer of system core to identify problem.

2.2 Reverse Engineering

Reverse Engineering is a process to evaluate source code from the prototype or the product itself. It is opposite to common used methodology for example like sdlc(system development life cycle) proceed from planning till maintenance in a sequence flow. Reverse engineering can be used to save time since it no need to start from the scratch but using final product as the analysis and measurement target. Greatly reduce cost and time waste when facing documentation missing or recover missing blueprint.



The screenshot displays the IDA View-A window, which is the main interface for viewing assembly code. The window is divided into several panes: 'IDA View-A' (the main pane), 'Hex View-1', 'Structures', and 'Enums'. The 'IDA View-A' pane shows assembly code for a segment named 'data'. The code includes comments and directives such as `segment dword public 'DATA' use64`, `assume cs:_data`, and `msg`. The 'Hex View-1' pane shows the corresponding hexadecimal representation of the code. The 'Structures' pane is empty, and the 'Enums' pane is also empty. The 'IDA View-A' pane has a status bar at the bottom that reads 'Synchronized with Hex View-1'.

```
.data:0000000000000000 ;  
.data:0000000000000000 ;  
.data:0000000000000000 ; This file was generated by The Interactive Disassembler (IDA)  
.data:0000000000000000 ; Copyright (c) 2022 Hex-Rays, <support@hex-rays.com>  
.data:0000000000000000 ; Freeware version  
.data:0000000000000000 ;  
.data:0000000000000000 ;  
.data:0000000000000000 ; Input SHA256 : F6B439EAE5F4025540A34F3F7DCA03639185A2AB782A3A8CE790A0C463796B9D  
.data:0000000000000000 ; Input MD5 : 850F0019505FB4D82EEAEAB7A3F78702  
.data:0000000000000000 ; Input CRC32 : E1FD437B  
.data:0000000000000000 ;  
.data:0000000000000000 ; File Name : \\wsl.localhost\Ubuntu\home\pbin\lab\lab1-7.o  
.data:0000000000000000 ; Format : ELF64 for x86-64 (Relocatable)  
.data:0000000000000000 ;  
.data:0000000000000000 ; Source File : 'lab1-7.asm'  
.data:0000000000000000 ;  
.data:0000000000000000 ; .686p  
.data:0000000000000000 ; .mmx  
.data:0000000000000000 ; .model flat  
.data:0000000000000000 ; .intel_syntax noprefix  
.data:0000000000000000 ;  
.data:0000000000000000 ;  
.data:0000000000000000 ;  
.data:0000000000000000 ; Segment type: Pure data  
.data:0000000000000000 ; Segment permissions: Read/Write  
.data:0000000000000000 ;  
.data:0000000000000000 _data segment dword public 'DATA' use64  
.data:0000000000000000 assume cs:_data  
.data:0000000000000000 msg db 41h ; A ; DATA XREF: .text:00000000000001A0  
.data:0000000000000001 db 20h  
.data:0000000000000002 db 42h ; B  
.data:0000000000000003 db 20h  
.data:0000000000000004 db 43h ; C  
.data:0000000000000005 db 0Ah  
.data:0000000000000005 _data ends  
.data:0000000000000005 ;  
.text:0000000000000010 ;  
.text:0000000000000010 ;  
.text:0000000000000010 ; Segment type: Pure code  
.text:0000000000000010 ; Segment permissions: Read/Execute  
.text:0000000000000010 _text segment para public 'CODE' use64  
.text:0000000000000010 assume cs:_text  
.text:0000000000000010 ;org 10h  
.text:0000000000000010 assume es:nothing, ss:nothing, ds:_data, fs:nothing, gs:nothing  
.text:0000000000000010 ;  
00000200 0000000000000000 .data:msg (Synchronized with Hex View-1)
```

Figure 1: IDA View (1)

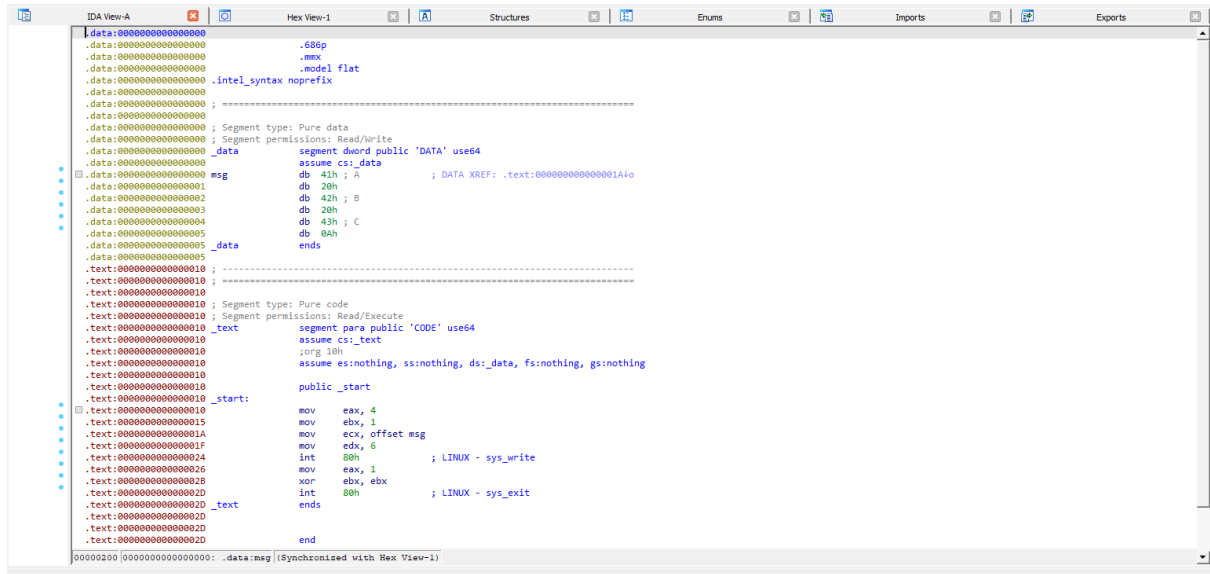


Figure 2: IDA View (2)

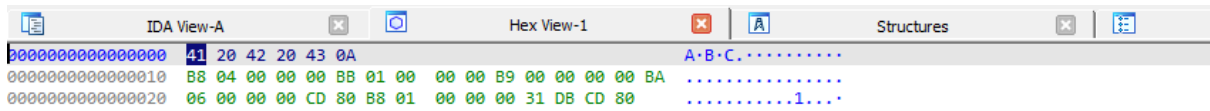


Figure 3 : Hex View

3.0 System Design using flowchart

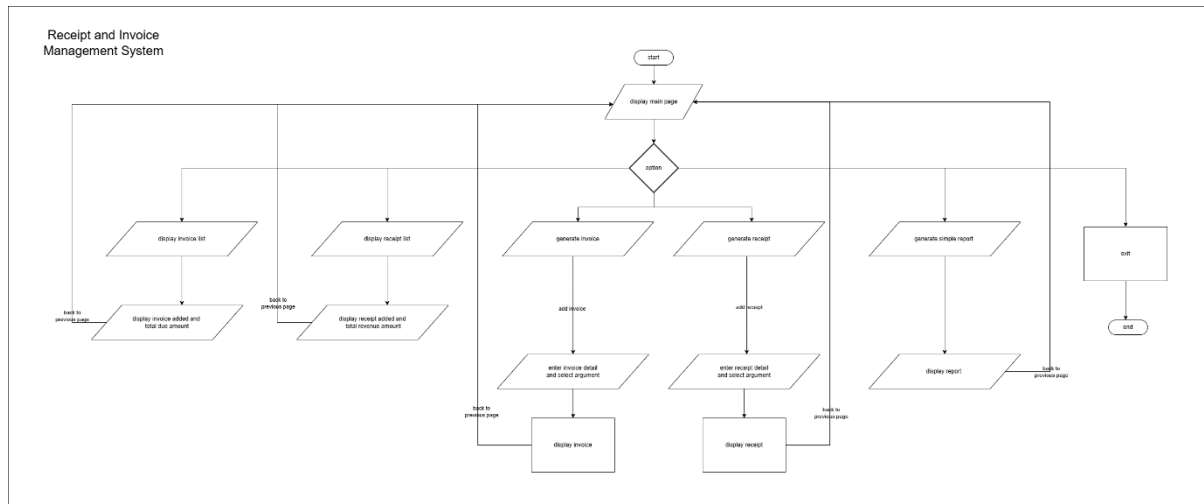


Figure 4: Flow Chart

The flow chart show the overview of the simple invoice and receipt management system. This chart mainly focus on navigation flow and design of the entire system. There are 6 valid menu option that can be chosen in the menu.

4.0 System Screenshot

```
=====
SIMPLE INVOICE & RECEIPT MGT SYSTEM
=====
1. Generate Receipt
2. Generate Invoice
3. View Receipt List
4. View Invoice List
5. Generate Simple Report
6. Exit

Enter your choice (1-6): 1
Enter Receipt Code: Rec00001
Enter Company Name: APU Sdn.Bhd
Enter Item Name: Printer
Enter Description: 10 items for office expansion
Enter Base Amount (Integer): 2000
Enter Discount Allowed (Integer): 100

=====
OFFICIAL RECEIPT
=====
Code       : Rec00001
Company    : APU Sdn.Bhd
Item       : Printer
Description: 10 items for office expansion
Base Amount : $2000
Discount   : $100

-----
Total Paid  : $1900
-----
```

Figure 5: Generate Receipt

```
=====
SIMPLE INVOICE & RECEIPT MGT SYSTEM
=====
1. Generate Receipt
2. Generate Invoice
3. View Receipt List
4. View Invoice List
5. Generate Simple Report
6. Exit

Enter your choice (1-6): 2
Enter Receipt Code: Inv00001
Enter Company Name: Cleaning Sdn.Bhd
Enter Item Name: Cleaning Services
Enter Description: Monthly factory cleaning
Enter Base Amount (Integer): 1000
Enter Discount Received (Integer): 0

=====
OFFICIAL INVOICE
=====
Code       : Inv00001
Company    : Cleaning Sdn.Bhd
Item       : Cleaning Services
Description : Monthly factory cleaning
Base Amount : $1000
Discount   : $0

-----
Total Due   : $1000
-----
```

Figure 6: Generate Invoice


```
=====
SIMPLE INVOICE & RECEIPT MGT SYSTEM
=====
1. Generate Receipt
2. Generate Invoice
3. View Receipt List
4. View Invoice List
5. Generate Simple Report
6. Exit

Enter your choice (1-6): 3

--- STORED RECEIPTS DIRECTORY ---
Code       : Rec00001
Total Paid  : $1900
-----
```

Figure 7: View Receipt List

```
=====
SIMPLE INVOICE & RECEIPT MGT SYSTEM
=====
1. Generate Receipt
2. Generate Invoice
3. View Receipt List
4. View Invoice List
5. Generate Simple Report
6. Exit

Enter your choice (1-6): 4

--- STORED INVOICES DIRECTORY ---
Code       : Inv00001
Total Due   : $1000
-----
```

Figure 8 : View Invoice List

```
=====
SIMPLE INVOICE & RECEIPT MGT SYSTEM
=====
1. Generate Receipt
2. Generate Invoice
3. View Receipt List
4. View Invoice List
5. Generate Simple Report
6. Exit

Enter your choice (1-6): 5

=====
SYSTEM REPORT
=====
Total Receipts Generated : 1
Total Invoices Generated : 1
Total Revenue (Receipts) : $1900
Total Owed (Invoices)    : $1000
```

Figure 9 : Generate Simple Report

```
=====
SIMPLE INVOICE & RECEIPT MGT SYSTEM
=====
1. Generate Receipt
2. Generate Invoice
3. View Receipt List
4. View Invoice List
5. Generate Simple Report
6. Exit

Enter your choice (1-6): 6

Shutting down system. Goodbye!
pbin@PBLapTop:~/ccllt/assignment$ |
```

Figure 10 : Exit

5.0 Source Code with explanation

```
; =====  
; Simple Invoice & Receipt Management System (NASM)  
; Ubuntu Linux (64-bit)  
; =====  
  
; --- Syscall Constants --- (not useful but look better)  
STDIN      equ 0  
STDOUT     equ 1  
SYS_READ   equ 0  
SYS_WRITE  equ 1  
SYS_EXIT   equ 60  
  
; --- Application Constants ---  
MAX_RECORDS equ 10  
STR_MAX     equ 32      ; Maximum length for string inputs  
REC_SIZE    equ 144     ; 32(Code) + 32(Company) + 32(Item) + 32(Desc) + 8(Amt) + 8(Disc)  
  
section .data
```

Figure 11: Predefine Value

At start of the code, system call service is assigned to make it look better. Application value is predefined for later calculation and storage use.

```
section .data  
; --- ANSI Color Codes for UI Formatting ---  
CLR_RST    db 27, "[0m", 0  
CLR_RED    db 27, "[31m", 0  
CLR_GRN    db 27, "[32m", 0  
CLR_YEL    db 27, "[33m", 0  
CLR_CYA    db 27, "[36m", 0  
CLR_BLD    db 27, "[1m", 0  
  
; --- Menu Strings ---  
menu_top    db 10, 27, "[1;36m", "===== ", 10  
            db "  SIMPLE INVOICE & RECEIPT MGT SYSTEM  ", 10  
            db "===== ", 27, "[0m", 10, 0  
  
m_opt1      db 27, "[32m", "1.", 27, "[0m", " Generate Receipt", 10, 0  
m_opt2      db 27, "[32m", "2.", 27, "[0m", " Generate Invoice", 10, 0  
m_opt3      db 27, "[32m", "3.", 27, "[0m", " View Receipt List", 10, 0  
m_opt4      db 27, "[32m", "4.", 27, "[0m", " View Invoice List", 10, 0  
m_opt5      db 27, "[32m", "5.", 27, "[0m", " Generate Simple Report", 10, 0  
m_opt6      db 27, "[31m", "6.", 27, "[0m", " Exit", 10, 0  
  
m_prompt    db 10, 27, "[1;33m", "Enter your choice (1-6): ", 27, "[0m", 0  
err_msg     db 27, "[1;31m", "Invalid choice. Please enter a number between 1 and 6.", 27, "[0m", 10, 0  
nl          db 10, 0
```

Figure 12: Section .Data (1)

```

; =====
; Receipt Generation and Viewing Logic
; =====

section .data
; --- Specific Prompts & UI Strings ---
p_code    db "Enter Receipt Code: ", 0
p_comp    db "Enter Company Name: ", 0
p_item    db "Enter Item Name: ", 0
p_desc    db "Enter Description: ", 0
p_amt     db "Enter Base Amount (Integer): ", 0
p_disc    db "Enter Discount Allowed (Integer): ", 0

msg_full   db 10, 27, "[1;31m", "Error: Maximum record limit reached (10/10).", 27, "[0m", 10, 0
msg_none   db 10, 27, "[1;33m", "No records found.", 27, "[0m", 10, 0

rec_head   db 10, 27, "[1;32m", "===== ", 10
           db "          OFFICIAL RECEIPT          ", 10
           db "===== ", 27, "[0m", 10, 0
rec_sep    db 27, "[36m", "----- ", 27, "[0m", 10, 0

lbl_code   db 27, "[1m", "Code      : ", 27, "[0m", 0
lbl_comp   db 27, "[1m", "Company   : ", 27, "[0m", 0
lbl_item   db 27, "[1m", "Item      : ", 27, "[0m", 0
lbl_desc   db 27, "[1m", "Description: ", 27, "[0m", 0
lbl_amt    db 27, "[1m", "Base Amount : $", 27, "[0m", 0
lbl_disc   db 27, "[1m", "Discount    : $", 27, "[0m", 0
lbl_tot    db 27, "[1;32m", "Total Paid : $", 0 ; Color reset handled dynamically

view_r_hd  db 10, 27, "[1;35m", "--- STORED RECEIPTS DIRECTORY ---", 27, "[0m", 10, 0

```

Figure 13: Section .Data (2)

Section .data is being used to define(byte) for variable which will be used for later. There are also color that being assigned first for later visualization to make it look more beautiful. There are multiple section .data is written in the code to arrange the code more structure and easier to adjust. Multiple section .data is created for different section like menu, receipt generation logic ... for easier management and adjustment.

```

section .bss
; --- Variables & Input Buffers ---
choice     resb 2           ; 1 byte for choice + 1 byte for newline

receipt_cnt resq 1          ; 64-bit integer tracking total receipts
invoice_cnt resq 1          ; 64-bit integer tracking total invoices

; --- Data Storage ---
; Each record takes REC_SIZE bytes. We will use pointers to navigate these in later steps.
receipts   resb REC_SIZE * MAX_RECORDS
invoices    resb REC_SIZE * MAX_RECORDS

```

Figure 14: Section .bss (1)

```

section .bss
; --- Temporary Conversion Buffer ---
tmp_buf     resb 32         ; Buffer for integer-to-string operations

```

Figure 15: Section .bss (2)

section .bss is being used to assign variable for buffer which will be used for receiving user input later. Storage for receipt and invoice data also being created in this section. Multiple section .bss is created for different use like menu choice and receipt generation part for easier management and adjustment.

```

section .text
    global _start

_start:
    ; Initialize our counters to zero
    mov qword [receipt_cnt], 0
    mov qword [invoice_cnt], 0

main_menu:
    ; Print Title
    mov rsi, menu_top
    call print_string

    ; Print Options
    mov rsi, m_opt1
    call print_string
    mov rsi, m_opt2
    call print_string
    mov rsi, m_opt3
    call print_string
    mov rsi, m_opt4
    call print_string
    mov rsi, m_opt5
    call print_string
    mov rsi, m_opt6
    call print_string

    ; Print Prompt
    mov rsi, m_prompt
    call print_string

    ; Read user choice
    mov rsi, choice
    mov rdx, 2
    call read_input

    ; --- Menu Branching Logic ---

```

Figure 16 : Section .text (1)

```

; -----
; str_to_int: Converts an ASCII string to a 64-bit integer.
; Input: rsi = Pointer to string
; Output: rax = Integer value
; -----
str_to_int:
    push rcx
    push rdx

    xor rax, rax            ; Clear rax (will hold our final integer)
    xor rcx, rcx           ; Clear rcx (index)

.next_digit:
    movzx rdx, byte [rsi + rcx] ; Load a single byte into rdx, zero-extending it

    cmp rdx, 0              ; Check for null terminator
    je .done
    cmp rdx, 10             ; Check for newline (just in case)
    je .done

    cmp rdx, '0'            ; If character is less than '0', it's not a digit
    jl .done
    cmp rdx, '9'            ; If character is greater than '9', it's not a digit
    jg .done

    sub rdx, '0'            ; Convert ASCII character to its integer value (e.g., '5' -> 5)

    imul rax, 10            ; Multiply current total by 10 (shift left in base 10)
    add rax, rdx            ; Add the new digit

    inc rcx                ; Move to next character
    jmp .next_digit

.done:
    pop rdx
    pop rcx
    ret

```

Figure 17 :Section .text (2)

Section .text is being use to write code, program, logic, arithmetic operation and loop. Jump, compare and call with different constrain are being widely used to accomplish wanted functionality and navigation flow. Since assembly language used stack as the data structure , stack function like pop and push also being widely used by in the program to modify the register value.

6.0 Conclusion

Low-level language can be used to design a simple system like this but it only has limited scalability, functionality and visualization. The system can't perform functionality like store the data into other file, memorize the past event due to NASM limitation. Low level language is faster and simpler in execution but not user friendly and lesser functionality. Conclusion, low level language is more suitable for program or system that no need good visualize and need high performance speed.

References

Attari, M. T., & Nawaz, M. A. (2024). *Importance of assembly language in cyber security and reverse engineering*. ResearchGate. <https://www.researchgate.net/publication/376170194>